LAB 5
STATIC CODE ANALYSIS

Pet Shelter Adoption Register

PES2UG24CS323
Nirav Choudhary
Section F

1. Objective

To improve Python code quality, security, and style by applying static analysis with Pylint, Bandit, and Flake8; documenting issues; fixing the code; and reflecting on the role and limitations of static analysis.

2. Tools and Results

Pylint checks code quality and likely programming errors. Bandit checks for common security weaknesses. Flake8 checks style and common lint issues.

Tool	Observed result	Interpretation
Pylint	Final report: 10.00/10 (previous run: 9.62/10, +0.38). The naming and global-statement warnings were addressed in the final code.	The final Pylint report rates the submitted version at 10.00/10.
Bandit	Latest pasted report: No issues identified; 0 low, 0 medium, 0 high issues.	No security findings were reported by this scan.
Flake8	Latest pasted report was empty (0 KB).	No Flake8 violations were reported by this run.

The final Pylint report supplied for this submission rates the code at 10.00/10. Bandit identified no issues, and the Flake8 report was empty.

3. Issues Identified and Fixes

Issue	Type	Original line(s)	Description / impact	Fix approach
SQL string interpolation	Security	Original line 10	User-provided name and species were inserted directly into SQL, allowing input to alter query syntax.	Replaced interpolation with parameterized SQL placeholders and a tuple of values.
Mutable default argument	Bug / maintainability	Original line 9	The default tags list could be shared across function calls.	Changed the default to None and create a new list inside the function when no list is supplied.
Bare except	Reliability	Original lines 26–28	A bare exception handler concealed the reason an operation failed and could hide programming errors.	Removed the broad handler and explicitly handle the empty query result.
Silent broad exception	Reliability	Original lines 33–35	Catching Exception and doing nothing could make a failed deletion appear successful.	Removed the silent handler; the function returns the number of deleted rows and unexpected database errors remain visible.
Oldest-pet query handling	Correctness	Original lines 26–28	An empty result could cause row indexing to fail;	Select only the required column, use LIMIT 1, and

			the query also fetched more data than needed.	return None if no available pet exists.
Adoption count updated unconditionally	Logic bug	Original lines 18–22	The counter could increase even if no pet row was changed or the pet was already adopted.	Validate the ID and increment only when exactly one row is updated.
Predictable adoption reference	Security / design	Original lines 24–25	The standard random generator is not designed for security-sensitive tokens.	Use the secrets module to generate the short adoption reference.
Inconsistent naming and formatting	Style	Original lines 9–38	CamelCase names and compressed formatting reduced consistency and readability.	Use snake_case names, readable formatting, docstrings, and type hints.
Missing input validation	Reliability	Original registration/adoption functions	Blank names/species and invalid ages or IDs could be accepted.	Validate names, species, ages, and adoption IDs; add database constraints.
Demo ran on import	Maintainability	Original main block	Importing the module immediately executed the demo.	Guard execution with if __name__ == '__main__'.

4. Summary of Code Improvements

- Parameterized SQL prevents input values from being interpreted as SQL syntax.
- Input validation rejects blank names/species and invalid ages or pet IDs.
- A None default prevents accidental reuse of a mutable list between calls.
- Broad and silent exception handling was removed so unexpected failures are visible.
- Adoption status and count change only when a database row is successfully updated.
- The oldest available pet lookup handles an empty shelter safely.
- The secrets module generates a less predictable adoption reference code.
- Function names, docstrings, type hints, logging, and the main guard improve readability and reuse.

5. Reflection Questions

5.1 What is static code analysis, and how is it different from testing?

Static code analysis inspects source code without executing the program. Pylint and Flake8 identify potential maintainability, correctness, and style problems, while Bandit looks for common security risks. Testing executes code with selected inputs to observe behaviour. Static analysis can flag risky code paths that a test never exercises, while tests help confirm the program's actual behaviour. They complement each other.

5.2 Which issues were most important, and why?

SQL string interpolation was important because user-controlled values could change the meaning of a database query. The mutable default argument and broad exception handlers were also important: shared state can make registrations interfere with one another, and swallowed exceptions can conceal failed operations. Incorrect adoption counting was a logic problem because it could report an adoption even when no record changed. These issues were prioritized over cosmetic formatting.

5.3 How did the fixes improve code quality, security, and maintainability?

Parameterized SQL separates instructions from data. Explicit validation rejects invalid records before they reach the database. A None default avoids sharing a list between calls, and removing broad silent exception handling makes unexpected failures visible. The adoption count now changes only when a database row is actually updated. Clear function names, docstrings, type hints, logging, and a main guard make the program easier to understand, reuse, and test.

5.4 What did you learn about Pylint, Bandit, and Flake8, and what are their limitations?

The tools serve different purposes: Pylint checks code quality and likely programming mistakes, Flake8 checks style and common lint errors, and Bandit highlights patterns associated with security weaknesses. Their findings need interpretation rather than blind acceptance; a warning is a clue, not proof of an exploitable vulnerability, and passing a tool does not

prove a program is correct or secure. Using the tools alongside functional tests gives broader coverage than relying on only one.

6. Running and Verifying the Program

```
python pet_shelter.py

pylint pet_shelter.py > pylint_report.txt

bandit -r pet_shelter.py > bandit_report.txt 2>&1

flake8 pet_shelter.py > flake8_report.txt
```

Final Python Source

```
"""Small in-memory animal shelter register used for Lab 5 static analysis.

The demo registers three pets, adopts one, prints the oldest available pet,
creates an adoption code, reports shelter status, and removes remaining pets.
"""

import logging
import secrets
import sqlite3
from typing import Optional

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
LOGGER = logging.getLogger(__name__)


def create_connection() -> sqlite3.Connection:
    """Create and initialize an in-memory SQLite database."""
    connection = sqlite3.connect(":memory:")
    connection.execute(
        """CREATE TABLE pets (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            species TEXT NOT NULL,
            age INTEGER NOT NULL CHECK (age >= 0),
            adopted INTEGER NOT NULL DEFAULT 0 CHECK (adopted IN (0, 1))
        )"""
    )
    return connection


conn = create_connection()
shelter_state = {"adoption_count": 0}


def register_pet(
    name: str, species: str, age: int, tags: Optional[list[str]] = None
) -> list[str]:
    """Register a pet safely and return its species tags.

    Parameterized SQL prevents user-provided strings from becoming SQL syntax.
    A fresh tag list is created for each call unless one is supplied.
    """
    if not isinstance(name, str) or not name.strip():
        raise ValueError("Pet name must be a non-empty string.")
    if not isinstance(species, str) or not species.strip():
        raise ValueError("Pet species must be a non-empty string.")
    if isinstance(age, bool) or not isinstance(age, int) or age < 0:
        raise ValueError("Pet age must be a non-negative integer.")

    pet_tags = [] if tags is None else tags
    conn.execute(
        "INSERT INTO pets (name, species, age, adopted) VALUES (?, ?, ?, 0)",
        (name.strip(), species.strip(), age),
    )
    pet_tags.append(species.strip())
    return pet_tags
```

```
def adopt(pet_id: int) -> bool:
    """Mark an available pet as adopted; return whether it was updated."""
    if isinstance(pet_id, bool) or not isinstance(pet_id, int) or pet_id <= 0:
        raise ValueError("Pet ID must be a positive integer.")

    cursor = conn.execute(
        "UPDATE pets SET adopted = 1 WHERE id = ? AND adopted = 0",
        (pet_id,),
    )
    if cursor.rowcount == 1:
        shelter_state["adoption_count"] += 1
        return True
    LOGGER.warning("Pet ID %s was not found or was already adopted.", pet_id)
    return False



def adoption_code() -> str:
    """Return a harder-to-guess short adoption reference code."""
    return f"PET-{secrets.randbelow(9000) + 1000}"



def oldest_available() -> Optional[str]:
    """Return the name of the oldest unadopted pet, or None if none exist."""
    row = conn.execute(
        """
        SELECT name
        FROM pets
        WHERE adopted = 0
        ORDER BY age DESC, id ASC
        LIMIT 1
        """
    ).fetchone()
    return None if row is None else str(row[0])



def clear_unadopted() -> int:
    """Delete unadopted pets and return the number of deleted rows."""
    cursor = conn.execute("DELETE FROM pets WHERE adopted = 0")
    return cursor.rowcount



def shelter_status() -> str:
    """Report whether the shelter has recorded any successful adoptions."""
    return "active" if shelter_state["adoption_count"] > 0 else "quiet"



def main() -> None:
    """Run a small demonstration of the shelter register."""
    register_pet("Bruno", "dog", 7)
    register_pet("Mitzi", "cat", 3)
    register_pet("Kiwi", "parrot", 12)
    adopt(1)
    LOGGER.info(
        "Oldest available: %s | Adoption code: %s | Status: %s",
        oldest_available(),
        adoption_code(),
        shelter_status(),
    )
    removed = clear_unadopted()
    LOGGER.info("Removed %s unadopted pet record(s).", removed)



if __name__ == "__main__":
    main()
```